Applying Token Reduction Strategies To Reduce Reasoning Inefficiency In Agentic LLM Systems

Avni Rajagopal

Introduction

THE PROBLEM: Agentic AI can't be scaled

- LLM systems with multiple agents, reasoning, task usage (agentic) → powerful for task completion, but requires long prompts and long outputs
- LLMs use tokens → more tokens = more cost, more processing time (quadratically) → agentic AI can't scale

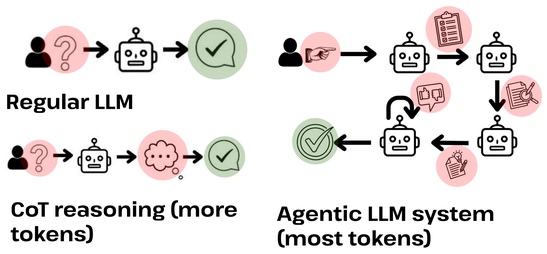


Figure 1. Various LLM applications and their outputs
Created by Avni Rajagopal using Canva

THE SOLUTION: token reduction

- Token reduction = any pruning from prompts and/or outputs
- Could maintain the benefits of agentic AI and reduce the cost
- Fewer tokens → agentic AI is more accessible

Background

- Token reduction strategies exist for all kinds of generative models—can decrease cost and improve performance
- AgentIF—a dataset to benchmark agentic systems on task completion via constraints
- **THE RESEARCH GAP:** Previous works investigate TR on non-agentic applications, or non-TR ways to improve agentic efficiency, but not TR on agentic systems
- **SHORTER CHAINS DO BETTER:** short-m@k, token reduction during reasoning LLM inference → found that excess tokens are unnecessary and possibly worsen performance
- **MULTI-AGENT SPECIFIC:** S^2-MAD, token reduction specific to multi-agent debate for reasoning → fewer tokens but little performance decline

Project Goal

I apply per-node prompt compression to a multi-agent LLM system that is agentic (task completion) rather than just reasoning (CoT or MAD) as previous works have done. With this novel framework, I investigate the amount of compression and types of compression possible before performance degrades.

The conclusions of this project can inform the design and scaling of agentic LLM systems in a variety of fields, since the benchmark used (AgentIF) covers agentic applications in several domains.

Materials

- Agentic LLM system used **GPT 4o mini** (context window 128k)
- Aux model for attention scores used **Qwen2.5 7B** (context window 131k) & **Llama-3.1-8B-Instruct** (context window 128k)
- Ran on **Jupyter Notebook** (Colab) with A100 GPU
- Used **AgentIF dataset**

Dataset

- AgentIF: a **2025 agentic benchmark**
- Instructions for agentic tasks in **several domains**
- **Constraints** associated with each instruction
- **Methods of evaluation** with code for each constraint
- Used **128 of 707 queries** for speed (stratified by prompt length + num constraints)

Setup

- **Built agentic LLM system:** planner → researcher → builder → critic
- Total of **nine experiments** with various configs
- Tested both **budget- and percent-based** span pruning
- Locally stored attention scores from an **open-source LLM**
- Prompt compression + intermediary compression at every node

Methodology

Prompt compression method

- Auxiliary LLM computed attention, weights from prompt text as keys and, for intermediary inputs, node-specific probe as queries
  - Compression asks: which spans are most important to the specific node?
- Used token-level scores, ranked and removed sentences to meet a budget or reach a sentence percentage kept

Token reduction experiment pipeline

- Local and runtime computation of attention
- Scores for AgentIF prompts stored locally → compressed based on budget/% and ran through system
- Node-specific compression on previous node's output

Control experiment pipeline

- Ran each query through the agentic LLM system without compressing prompt or any intermediary outputs

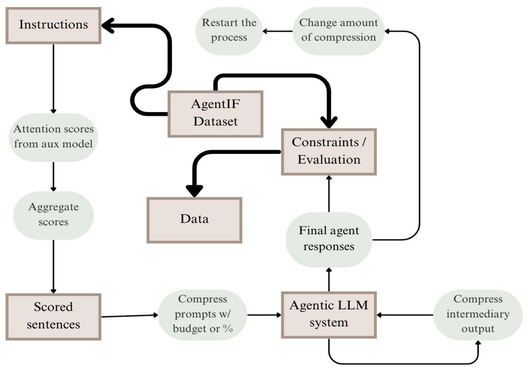


Figure 2. Flow of token reduction experiments
Created by Avni Rajagopal using Canva

DATA COLLECTED

- **CSR (constraint success rate):** proportion of all constraints were met
- **ISR (instruction success rate):** proportion of all instructions for which all constraints were met
- **Mean length of text passed to LLMs** (input, plan, research, build, and critique) in characters

Results

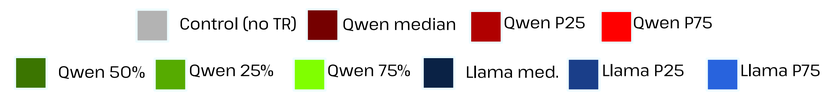
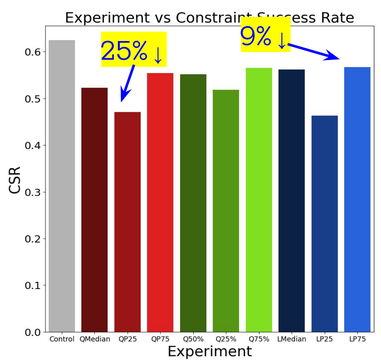


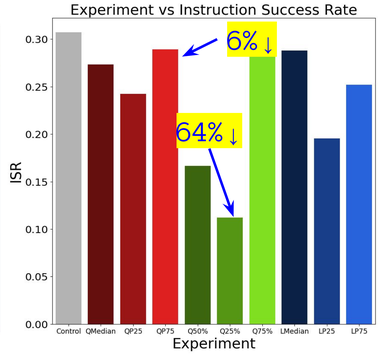
Figure 3. CSR graph
Created by Avni Rajagopal



Constraint Success Rate

- Median vs P75, 50%, vs 75% very similar → can use median or 50% without losing significant performance on CSR
- Llama and Qwen functionally equal

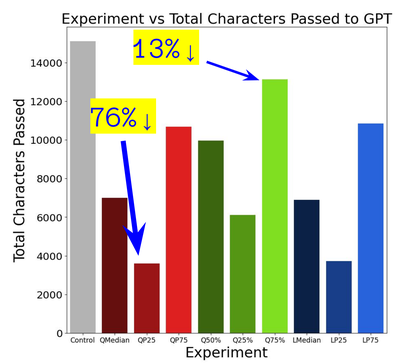
Figure 4. ISR graph
Created by Avni Rajagopal



Instruction Success Rate

- Budget-based meaningfully outperformed %-based
- 75% scored higher than 50% – those 25% spans matter, sensitive
- Llama–median scored higher than P75

Fig 5. Mean chars passed
Created by Avni Rajagopal



Mean chars passed

- Best case 76% reduction w/ control–large savings
- Budget-based compression is cheaper

Conclusions

- Prompt compression **significantly decreased average total length passed to GPT**
- Performance drops, but even less aggressive compression saves
- **25th percentile budget compression** is cheapest (76% reduction in length) but also 25% CSR reduction, 21% ISR reduction
- Median budget **similar performance to** 75th percentile, but **much cheaper**
- For CSR, percentage-based **marginally better** than budget-based, but for ISR, budget **notably better** than percentage → choose budget
- **Better performance not always greater length**—sometimes more compression → equal or better results

Takeaways

MORE TOKENS ≠ BETTER RESULTS
- What matters more is which spans are removed

PERFORMANCE VS COST
- Token reduction **drops performance** on agentic tasks, but reduces length passed across the board, **decreasing cost**

WHICH CONFIGS WIN?
- Budget-based > percentage-based compression
- Median budgeting or 25th percentile budgeting are most promising

So what?

LLMs like GPT are **priced per token** + # input tokens **affect latency →** fewer tokens directly **reduces cost and time needed** to run the agentic system. Cost/latency improvements make agentic AI **more accessible and scalable,** allowing it to be applied on larger scales.

Future Research

- Work to decrease performance gap while maintaining length gap by fine-tuning TR
- Test more fine-grained degrees of TR to see where performance drops (sweet spot)
- Try different aux/agent models–does this impact the efficiency of TR?
- Compare TR methods specific to / designed for agentic systems (similar to S^2-MAD)
- See what types of spans impact performance – what about a sentence makes it more important to an agentic application?

References

1. Qi, Y., et al. (2025). AGENTIF: Benchmarking Instruction Following of Large Language Models in Agentic Scenarios. *arXiv*. https://arxiv.org/abs/2505.16944
2. Kong, Z., et al. (2025). Token Reduction Should Go Beyond Efficiency in Generative Models -- From Vision, Language to Multimodality. *arXiv*. https://arxiv.org/abs/2505.18227
3. Hassid, M., et al. (2025). Don't Overthink it. Preferring Shorter Thinking Chains for Improved LLM Reasoning. *ArXiv*. https://arxiv.org/abs/2505.17813
4. Zeng, Y., et al. (2025). S2-MAD: Breaking the Token Barrier to Enhance Multi-Agent Debate Efficiency. *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, 1, 9393–9408.* https://doi.org/10.18653/v1/2025.naacl-long.475
5. Dong, P., et al. (2025). Can Compressed LLMs Truly Act? An Empirical Evaluation of Agentic Capabilities in LLM Compression. *Proceedings of the 42nd International Conference on Machine Learning*, in *Proceedings of Machine Learning Research 267:14169-14202.* https://proceedings.mlr.press/v267/dong25k.html.
6. Some of the code for this project was generated using generative AI (GPT-5.2).